

ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ
ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ
ΗΛΕΚΤΡΟΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ
ΡΟΗ Λ, ΠΡΟΧΩΡΗΜΕΝΑ ΘΕΜΑΤΑ ΒΑΣΕΩΝ ΔΕΔΟΜΕΝΩΝ



Αναφορά
Εξαμηνιαίας Εργασίας
Group 26

Μέλη:
Γκίνη Δήμητρα 03120128
Θάνου Παρασκευή 03120225

QUERY 1

Υλοποίηση του Query 1 χρησιμοποιώντας DataFrame και RDD APIs και σύγκριση απόδοσης.

Τρόπος υλοποίησης:

Για να ταξινομήσουμε τις ηλικιακές ομάδες των θυμάτων σε περιστατικά που περιλαμβάνουν “βαριά σωματική βλάβη”, αρχικά φορτώσαμε τα δεδομένα εγκληματικότητας από τα δυο αρχεία που μας δίνονται και αφού τα φιλτράραμε ώστε να πληρούν τα ηλικιακά όρια κάθε ομάδας και την προϋπόθεση για βαριά σωματική βλάβη, τα ενοποιήσαμε σε ένα ενιαίο σύνολο δεδομένων και τα ταξινομήσαμε κατάλληλα.

Αποτελέσματα και Σχόλια:

Και με τις δύο υλοποιήσεις βρήκαμε τον ίδιο αριθμό εγγραφών σε κάθε ηλικιακή ομάδα. Ωστόσο, παρατηρήσαμε πολύ μεγάλη διαφορά στην επίδοση τους, αφού η υλοποίηση με RDD χρειάστηκε 49.58 sec ενώ η υλοποίηση με DataFrame μόλις 0.21 sec. Η διαφορά αυτή στην επίδοση ήταν αναμενόμενη.

Τα Resilient Distributed Datasets (RDDs) είναι ο βασικός τρόπος διαχείρισης δεδομένων (low-level) στο Apache Spark. Πρόκειται για μια κατανεμημένη δομή δεδομένων που επιτρέπει την επεξεργασία μεγάλων όγκων δεδομένων σε παράλληλα clusters. Τα RDDs ανακτούν δεδομένα σε περίπτωση σφαλμάτων (Resilient), είναι κατανεμημένα (distributed) σε διάφορα nodes και αμετάβλητα (immutable), καθώς δεν μπορούν να τροποποιηθούν μετά τη δημιουργία τους. Υποστηρίζουν lazy evaluation (υπολογισμός κατόπιν εντολής), επιτρέποντας τη βελτιστοποίηση των υπολογισμών.

Τα DataFrames στο Apache Spark είναι ένας εναλλακτικός τρόπος διαχείρισης δεδομένων, high-level, που παρέχει δομημένα δεδομένα σε μορφή πίνακα (όπως σε βάσεις δεδομένων ή πίνακες Excel). Υποστηρίζουν SQL queries και προσφέρουν βελτιστοποίηση μέσω του Catalyst Optimizer, ο οποίος αναλύει και βελτιστοποιεί τα ερωτήματα για μεγαλύτερη απόδοση. Τα DataFrames επιτρέπουν την παράλληλη επεξεργασία δεδομένων και υποστηρίζουν lazy evaluation, όπως τα RDDs, αλλά με πιο αποδοτικό τρόπο.

Καθώς, τα RDD είναι low-level έχουν κάποια αρνητικά σε σχέση με τα DataFrames όπως ότι είναι πιο αργά σε γλώσσες όπως η python που χρησιμοποιήσαμε, που δεν έχουν JVM. Επιπλέον, δεν έχουν αυτόματο optimization και έτσι το Spark δεν γνωρίζει την πράξη ή τον τύπο που τρέχει, σε αντίθεση με τα DataFrames, που λόγω του Catalyst Optimizer στον οποίο βασίζονται, βελτιστοποιείται αυτόματα η απόδοσή τους.

QUERY 2

Τρόπος υλοποίησης:

Για τον εντοπισμό των 3 Αστυνομικών Τμημάτων με το υψηλότερο ποσοστό κλεισμένων (περατωμένων) υποθέσεων ανά έτος, αρχικά φορτώσαμε τα δεδομένα εγκληματικότητας από τα δυο αρχεία που μας δίνονται και τα ενοποιήσαμε σε ένα ενιαίο σύνολο δεδομένων. Στη συνέχεια, επιλέχθηκαν οι στήλες που περιείχαν την ημερομηνία του συμβάντος, το όνομα του τμήματος και την κατάσταση της υπόθεσης, ενώ εξήχθη η χρονιά από την ημερομηνία. Για τον υπολογισμό του ποσοστού κλεισμένων υποθέσεων, φιλτράραμε τις κλεισμένες υποθέσεις, οι οποίες ήταν αυτές που είχαν καταστάσεις : "Adult Arrest", "Adult Other", "Juv Arrest" και "Juv Other". Ακολούθως, για κάθε έτος και τμήμα, υπολογίστηκαν ο αριθμός των κλεισμένων υποθέσεων και το σύνολο των υποθέσεων, και στη συνέχεια το ποσοστό κλεισμένων υποθέσεων. Για την ανάδειξη των 3 κορυφαίων τμημάτων ανά έτος, χρησιμοποιήθηκε Window για την κατάταξη των τμημάτων με βάση το ποσοστό κλεισμένων υποθέσεων σε φθίνουσα σειρά και επιλέχθηκαν μόνο τα τμήματα με κατάταξη έως και 3. Τέλος, τα αποτελέσματα ταξινομήθηκαν με βάση το έτος και την κατάταξη, και αποδόθηκαν τα 3 τμήματα με την καλύτερη απόδοση για κάθε έτος. Ο τελικός πίνακας υπάρχει στον τρεγμένο κώδικα του project μας.

α) ΥΛΟΠΟΙΗΣΗ ΜΕ DATAFRAMES

SQL API (analytika)

Υλοποιήσαμε το 2ο query αρχικά χρησιμοποιώντας DataFrames και ο χρόνος εκτέλεσης βγήκε 18.04 sec.

Στην συνέχεια υλοποιήσαμε με SQL API και ο χρόνος εκτέλεσης είναι 22.72 sec.

Το γεγονός αυτό οφείλεται στο ότι το API των DataFrames χρησιμοποιεί τον Catalyst Optimizer της Spark για να βελτιστοποιήσει αυτόματα τα ερωτήματα.

Επιπλέον, τα SQL queries περνούν από ένα επιπλέον στάδιο επεξεργασίας, που μπορεί να προσθέσει μικρή καθυστέρηση. Με άλλα λόγια, τα SQL ερώτημα πρέπει πρώτα να αναλυθούν (parse), να μετατραπούν σε ένα σχέδιο εκτέλεσης (execution plan), και στη συνέχεια να εκτελεστούν από την Spark Engine.

β) ΣΥΓΚΡΙΣΗ CSV ΚΑΙ PARQUET

Επιλέξαμε την υλοποίηση με DATAFRAMES για την σύγκριση των χρόνων με parquet και με csv. Τα αποτελέσματα ήταν τα εξής:

Όταν τα δεδομένα εισάγονται σαν .csv: 19.71 sec.

Όταν τα δεδομένα εισάγονται σαν .parquet: 6.98 sec.

Παρατηρούμε ότι το parquet format ήταν σημαντικά γρηγορότερο σε σχέση με το csv. Ειδικότερα, οδήγησε σε βελτίωση του χρόνου εκτέλεσης κατά 12.72 sec.

Αυτό είναι κάτι που περιμέναμε να συμβεί αφού, το Parquet αποθηκεύει τα δεδομένα ανά στήλη (columnar storage), ενώ το CSV αποθηκεύει τα δεδομένα ανά γραμμή (row-based storage). Έτσι, όταν εκτελούμε ένα ερώτημα για το οποίο χρειαζόμαστε συγκεκριμένες στήλες, με το Parquet διαβάζονται μόνο οι απαιτούμενες στήλες από τον δίσκο, ενώ με το CSV, διαβάζονται όλες οι στήλες, ακόμα κι αν χρειαζόμαστε μόνο κάποιες από αυτές. Επομένως, σε αυτό το query όπου χρειαστήκαμε μόνο τις στήλες DATE OCC, AREA NAME και Status Desc, το parquet εξοικονόμησε πολύ χρόνο αποφεύγοντας το διάβασμα «άχρηστων στηλών».

QUERY 3

Τρόπος υλοποίησης:

Σε αυτό το ερώτημα, υπολογίσαμε το μέσο εισόδημα ανά άτομο και το μέσο ποσοστό εγκληματικότητας ανά άτομο για κάθε περιοχή συνδυάζοντας δεδομένα εισοδήματος, πληθυσμού και εγκληματικότητας. Αρχικά, προετοίμασα το dataset εισοδήματος εξάγοντας τα εισοδήματα και τις περιοχές, ενώ για το dataset πληθυσμού επεξεργάστηκα τα δεδομένα για να υπολογίσω τον συνολικό πληθυσμό και τα νοικοκυριά ανά zip code και περιοχή. Υπολόγισα το μέσο μέγεθος νοικοκυριού ανά περιοχή χρησιμοποιώντας έναν απλό μέσο όρο. Στη συνέχεια, ένωσα τα δεδομένα εισοδήματος και πληθυσμού για να υπολογίσω το μέσο εισόδημα ανά άτομο για κάθε περιοχή, βασιζόμενος στο συνολικό εισόδημα και τον πληθυσμό της κάθε περιοχής. Παράλληλα, επεξεργάστηκα τα δεδομένα εγκληματικότητας και γεωγραφικών πληροφοριών για να συνδέσω τα σημεία εγκλημάτων με τις περιοχές. Υπολόγισα τον ρυθμό εγκληματικότητας ανά άτομο για κάθε περιοχή και πήρα τον μέσο όρο αυτών των τιμών.

Τέλος, συνδύασα τα δεδομένα εισοδήματος και εγκληματικότητας σε έναν τελικό πίνακα, ο οποίος περιέχει για κάθε περιοχή το μέσο εισόδημα ανά άτομο και το μέσο ποσοστό εγκληματικότητας ανά άτομο, ταξινομημένα αλφαβητικά.

Στρατηγικές Join του Spark

Το Spark υποστηρίζει διαφορετικές στρατηγικές join, κάθε μία από τις οποίες είναι κατάλληλη για συγκεκριμένες περιπτώσεις:

1. **BroadcastHashJoin:**

Χρησιμοποιείται όταν μία από τις πηγές δεδομένων είναι αρκετά μικρή για να χωρέσει στη μνήμη. Τα δεδομένα της μικρότερης πηγής μεταδίδονται (broadcast) σε όλα τα nodes. Έτσι, κάθε executor μπορεί να εκτελέσει το join τοπικά, χρησιμοποιώντας το broadcasted dataset, χωρίς να χρειαστεί να ανταλλάξει δεδομένα με άλλους κόμβους.

2. **SortMergeJoin:**

Αυτή η μέθοδος είναι ιδανική για μεγάλα σύνολα δεδομένων που είναι ταξινομημένα ή μπορούν να ταξινομηθούν αποδοτικά. Χρησιμοποιεί συγχώνευση (merge) στα δεδομένα που έχουν ταξινομηθεί στις στήλες του join. Είναι ιδανικό για μεγάλα σύνολα δεδομένων που δεν χωρούν στη μνήμη.

3. **ShuffleHashJoin:**

Εφαρμόζεται όταν τα δεδομένα δεν είναι ταξινομημένα και δεν μπορούν να γίνουν broadcast. Η βασική του αρχή είναι να διαμερίσει (partition) και να αναδιανείμει (shuffle) τα δεδομένα από τους δύο πίνακες που συμμετέχουν στη σύζευξη, έτσι ώστε τα δεδομένα με την ίδια κλειδαριά σύζευξης (join key) να βρίσκονται στην ίδια διαμέριση.

4. **ShuffleReplicateNL:**

Χρησιμοποιείται ως έσχατη λύση όταν οι άλλες στρατηγικές δεν είναι αποδοτικές. Σε αυτόν τον αλγόριθμο, ο ένας πίνακας αναπαράγεται σε όλες τις διαμερίσεις του άλλου πίνακα, και στη συνέχεια εκτελείται το join τοπικά σε κάθε διαμέριση. Υλοποιεί nested loop join, αλλά είναι πολύ δαπανηρή σε πόρους.

5. **RangeJoin:**

Ιδανικό όταν τα δεδομένα είναι ταξινομημένα και η σύγκριση γίνεται σε εύρος τιμών, όπως αριθμητικά ή χρονολογικά δεδομένα. Είναι πιο αποδοτικό από τις υπόλοιπες στρατηγικές όταν τα δεδομένα είναι καλά οργανωμένα.

Ανάλυση των Join

1. **Joined_df**

Το πρώτο join έγινε μεταξύ του dataset aggregated_population_df, που περιλαμβάνει πληθυσμιακά δεδομένα για κάθε περιοχή (5 στήλες), και του income_split_df, που περιέχει 3 στήλες: τον ταχυδρομικό κώδικα (income_zip), την περιοχή (area_income), και το μέσο εισόδημα ανά νοικοκυριό (income).

Το income_split_df είναι μικρότερο από το πρώτο dataset, καθώς περιλαμβάνει λιγότερες στήλες και γραμμές.

Σύμφωνα με το explain, η στρατηγική που επιλέχθηκε ήταν **BroadcastHashJoin**.

Αυτό σημαίνει ότι το Spark αναγνώρισε το income_split_df ως μικρό dataset και το μετέδωσε (broadcast) σε όλα τα nodes. Η στρατηγική αυτή ήταν η βέλτιστη για το συγκεκριμένο join, καθώς ελαχιστοποιεί το κόστος μετακίνησης δεδομένων.

2. Joined_df1

Το δεύτερο join έγινε μεταξύ του dataset df1, που περιλαμβάνει γεωμετρικά δεδομένα των εγκλημάτων και του flattened_df, το οποίο περιέχει επίπεδα δεδομένα για τα οικοδομικά τετράγωνα του Los Angeles.

Σύμφωνα με το explain, η στρατηγική που επιλέχθηκε για αυτό το join ήταν **RangeJoin**. Αυτό σημαίνει ότι τα δεδομένα ήταν ταξινομημένα με τέτοιο τρόπο ώστε το Spark επέλεξε να χρησιμοποιήσει έναν **εύρος join (range join)** για να κάνει τη σύγκριση. Το **RangeJoin** είναι πιο αποδοτικό όταν τα δεδομένα έχουν ταξινομηθεί ή είναι ήδη σε διατεταγμένη σειρά, διευκολύνοντας τη διαδικασία του join.

Η στρατηγική αυτή είναι βέλτιστη όταν τα δεδομένα είναι ταξινομημένα κατά το πεδίο του join και μπορεί να μειώσει την ανάγκη για σύγκριση όλων των πιθανών ζευγών.

3. Το τρίτο join έγινε μεταξύ του dataset aggregated_income_population_df, το οποίο περιλαμβάνει τα δεδομένα πληθυσμού και εισοδήματος για κάθε περιοχή, και του final_crime_rate_per_area_per_person, το οποίο περιέχει τα ποσοστά εγκληματικότητας για κάθε περιοχή ανά άτομο.

Και τα δύο datasets περιέχουν μια κοινή στήλη "area" για το join.

Το dataset aggregated_income_population_df είναι μεγαλύτερο από το final_crime_rate_per_area_per_person, καθώς περιέχει πληθυσμιακά δεδομένα, ενώ το δεύτερο dataset περιλαμβάνει μόνο τα ποσοστά εγκληματικότητας για κάθε περιοχή.

Σύμφωνα με το explain, η στρατηγική που επιλέχθηκε για αυτό το join ήταν **SortMergeJoin**. Αυτή η στρατηγική χρησιμοποιείται όταν τα δεδομένα είναι ταξινομημένα και το join μπορεί να γίνει πιο αποτελεσματικά μέσω συγχώνευσης (merge) των δύο datasets.

Το **SortMergeJoin** είναι πιο αποδοτικό όταν τα δεδομένα είναι ήδη ταξινομημένα με βάση το πεδίο του join, επιτρέποντας τη συγχώνευση των δύο πινάκων χωρίς την ανάγκη πρόσθετων υπολογισμών.

Η στρατηγική αυτή είναι κατάλληλη για αυτό το join, δεδομένου ότι τα δεδομένα είναι ταξινομημένα κατά την περιοχή, και η συγχώνευση μπορεί να γίνει με μεγάλη αποδοτικότητα.

ΔΙΑΦΟΡΕΤΙΚΕΣ ΣΤΡΑΤΗΓΙΚΕΣ JOIN

Τώρα, με την χρήση του hint, θα επιβάλλω διαφορετικές στρατηγικές στο πρώτο join, για να δω ποια μέθοδος είναι πιο αποτελεσματική.

Θα μετρήσω τον χρόνο εκτέλεσης που απαιτεί η κάθε στρατηγική για την εκτέλεση του join.

Από τα αποτελέσματα βλέπουμε ότι ο καλύτερος τρόπος ήταν ο **ShuffleReplicateNL**.

Παρατηρώ ότι η πιο αποδοτική στρατηγική δεν ταυτίζεται με αυτή που έκανε ο catalyst optimizer. Το γεγονός αυτό έχει να κάνει με τη φύση και την κατανομή των δεδομένων.

Θα δοκιμάσω να ταξινομήσω τους πίνακες πριν τους κάνω join για να δω τι επίδραση θα έχει αυτό στους χρόνους των queries.

Ταξινομώ το `income_split_df` και το `population_df` ως προς το `zip` όπου θα γίνει και το join για να δω τι διαφορά θα επιφέρει. Και πάλι, η καλύτερη στρατηγική ήταν η **ShuffleReplicateNL**, η οποία σημείωσε τον μικρότερο χρόνο.

QUERY 4

Τρόπος υλοποίησης:

Χρησιμοποιώντας τις 3 περιοχές με το υψηλότερο και τις 3 περιοχές με το χαμηλότερο εισόδημα με τη βοήθεια του κώδικα από το query 3, και φιλτράροντας κατάλληλα τα δεδομένα εγκληματικότητας ώστε να αφορούν μόνο το έτος 2015 (για τον λόγο αυτή χρησιμοποιήσαμε μόνο το ένα από τα δύο αρχεία) βρήκαμε το φυλετικό προφίλ των θυμάτων, αφού το συνδέσαμε με το κατάλληλο αρχείο δεδομένων, και τα ταξινομήσαμε από το υψηλότερο στο χαμηλότερο.

Αποτελέσματα και Σχόλια:

1o configuration (2 executors 1 cores/2GB μνήμη)

Execution time = 83.48 seconds

2o configuration (2 executors 2 cores/4GB μνήμη)

Execution time = 64.38 seconds

3o configuration (2 executors 4 cores/8GB μνήμη)

Execution time = 59.17 seconds

Όπως είναι αναμενόμενο ο χρόνος εκτέλεσης με το πρώτο configuration, 1 core και 2GB μνήμης, είναι αρκετά αυξημένος καθώς έχουμε έναν core ανά executor, δηλαδή ο executor επεξεργάζεται ένα task τη φορά, περιορίζοντας σημαντικά την παραλληλοποίηση.

Αντίθετα, με την χρήση περισσότερων πόρων, δηλαδή 2 και 4 cores με 4 και 8 GB μνήμης στο 2o και 3o configuration αντίστοιχα, παρατηρείται σαφής βελτίωση στην απόδοση. Ο αυξημένος παραλληλισμός επιτρέπει την ταχύτερη εκτέλεση των tasks, μειώνοντας τον συνολικό χρόνο επεξεργασίας.

QUERY 5

Τρόπος υλοποίησης:

Σε αυτό το query, υλοποιήσαμε μια διαδικασία με Dataframes για να υπολογίσουμε τον αριθμό εγκλημάτων που σημειώθηκαν πλησιέστερα σε κάθε αστυνομικό τμήμα και τη μέση απόσταση αυτών των εγκλημάτων από το συγκεκριμένο τμήμα. Αρχικά, φορτώσαμε τα δεδομένα των αστυνομικών τμημάτων από ένα αρχείο CSV και προσθέσαμε στήλες συντεταγμένων γεωγραφικού μήκους (X) και πλάτους (Y) με ακρίβεια 10 δεκαδικών. Στη συνέχεια, δημιουργήσαμε γεωμετρικά σημεία για τις τοποθεσίες των τμημάτων και αντίστοιχα για τα εγκλήματα. Συσχετίσαμε κάθε έγκλημα με όλα τα τμήματα μέσω διασταυρούμενης σύζευξης (crossJoin), υπολογίσαμε την απόσταση μεταξύ των γεωγραφικών σημείων κάθε εγκλήματος και τμήματος (σε χιλιόμετρα) και διατηρήσαμε την πλησιέστερη αντιστοιχία για κάθε έγκλημα, χρησιμοποιώντας παραθυρικές συναρτήσεις (window functions). Τέλος, ομαδοποιήσαμε τα δεδομένα ανά τμήμα, υπολογίζοντας τον αριθμό των εγκλημάτων και τη μέση απόσταση από αυτά, και ταξινομήσαμε τα αποτελέσματα με φθίνουσα σειρά βάσει του αριθμού των εγκλημάτων.

Στην συνέχεια, δοκιμάσαμε διαφορετικά configurations και παρατηρήσαμε την επίδραση που είχαν στον χρόνο εκτέλεσης του query.

Παραθέτουμε τα αποτελέσματα παρακάτω:

1. 1st configuration
Execution time = 0.56 seconds
2. 2nd configuration
Execution time = 0.61 seconds.
3. 3rd configuration
Execution time = 0.58 seconds.

Το Spark διανέμει τις εργασίες σε πολλούς executors, και κάθε executor εκτελεί υπολογισμούς σε ξεχωριστούς πόρους.

Ο αριθμός των πυρήνων καθορίζει πόσες εργασίες μπορεί να εκτελεί ταυτόχρονα κάθε executor. Περισσότεροι πυρήνες ανά executor συνήθως σημαίνουν μεγαλύτερο παραλληλισμό, κάτι που μπορεί να επιταχύνει τον υπολογισμό.

Η μνήμη είναι κρίσιμος παράγοντας, καθώς καθορίζει πόσα δεδομένα μπορούν να αποθηκευτούν ή να υπολογιστούν χωρίς να απαιτείται αποθήκευση σε δίσκο. Περισσότερη μνήμη επιτρέπει πιο

αποτελεσματικούς υπολογισμούς, καθώς χρειάζεται λιγότερο χρόνο για να γράψει δεδομένα στον δίσκο.

Παρατηρούμε ότι το πρώτο configuration επιτυγχάνει τον μικρότερο χρόνο εκτέλεσης. Αυτό συμβαίνει διότι η μεγαλύτερη μνήμη ανά executor (8GB) επιτρέπει την επεξεργασία περισσότερων δεδομένων στη μνήμη και οι 4 πυρήνες ανά executor επιτρέπουν την εκτέλεση πολλαπλών εργασιών ταυτόχρονα.

Στο δεύτερο configuration, η αύξηση του αριθμού των executors και η μείωση των πυρήνων και της μνήμης ανά executor διασπά την εργασία σε μικρότερα τμήματα. Ωστόσο, αυτό αυξάνει την ανάγκη για επικοινωνία μεταξύ των εκτελεστών και μπορεί να επιβαρύνει το δίκτυο. Η διαχείριση της μνήμης σε αυτό το configuration ίσως είναι λιγότερο αποτελεσματική, αλλά η διαφορά στον χρόνο εκτέλεσης είναι μικρή.

Τέλος, η 3^η διαμόρφωση είναι η πιο αργή εξαιτίας της μικρής μνήμης και έχει τον λιγότερο παραλληλισμό (1 πυρήνας ανά executor). Γιαυτό, ο χρόνος εκτέλεσης είναι ο μεγαλύτερος.

LINK ΓΙΑ ΚΩΔΙΚΑ:

<https://github.com/DimitraGkini123/Advanced-databases-semester-project.git>